

JC4003 Natural Language Processing

Lecture 7: Logistic Regression

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

Logistic Regression

Logistic Regression Example

Supervised Classification

Logistic Regression

Logistic Regression

Logistic Regression is a statistical method used for binary classification that models the probability of a binary outcome based on one or more predictor variables.

- Important analytic **tool in natural and social sciences**.
- **Baseline supervised machine learning** tool for classification.
- Is also **the foundation of neural networks**.

Generative vs. Discriminative Classifiers

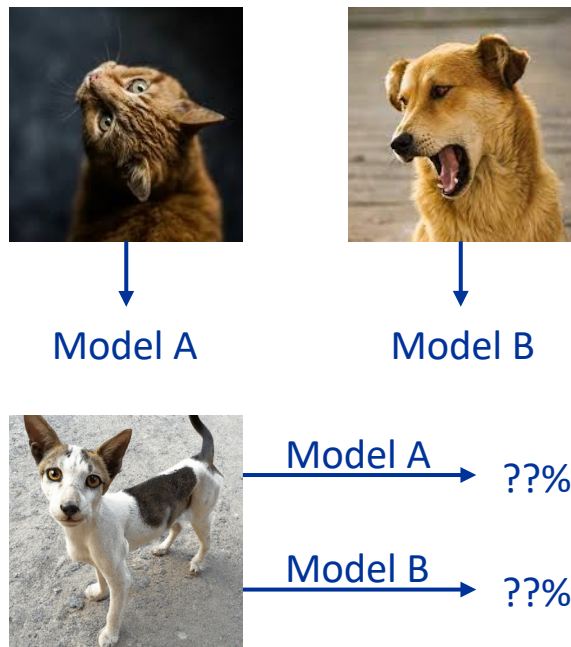
Generative Classifier:

1. Build model **A** for cat images
2. Learn about whiskers, ears, eyes
3. Assign probabilities to any image
4. Predict how cat-like is this picture

Also build another model **B** for dog images.

Now given a new image:

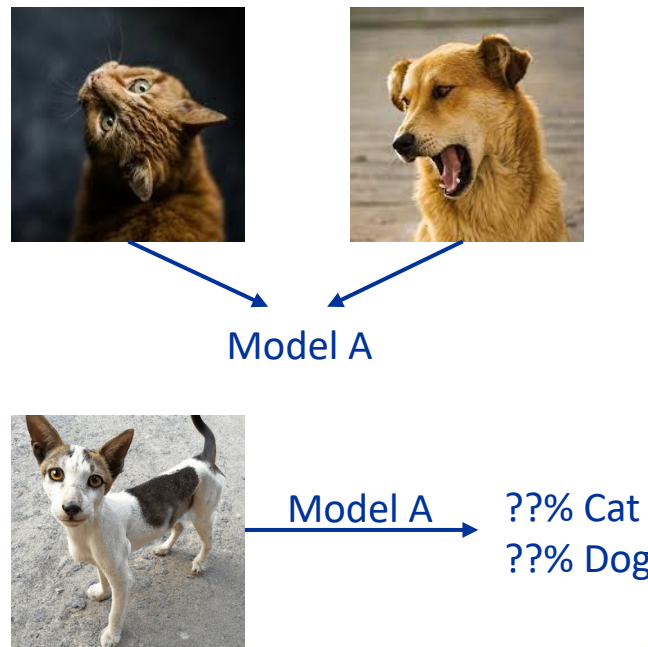
- Run both models and see which one fits better



Generative vs. Discriminative Classifiers

Discriminative Classifier:

A Discriminative Classifier uses **one model** that directly estimates the decision boundary or conditional probability of the target variable given the input features, **focusing on distinguishing between different classes.**



Naïve Bayes vs. Logistic Regression

Naïve Bayes -- Generative Classifier

$$\hat{c} = \underset{c \in \mathcal{C}}{\operatorname{argmax}} \underbrace{P(d|c)}_{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

Logistic Regression -- Discriminative Classifier

$$\hat{c} = \underset{c \in \mathcal{C}}{\operatorname{argmax}} \underbrace{P(c|d)}_{\text{posterior}}$$

Text Classification: Definition (recall)

Input:

- a document d
- a fixed set of classes $\mathcal{C} = \{c_1, c_2, \dots, c_j\}$

Output:

- a predicted class $c \in \mathcal{C}$

Binary Classification in Logistic Regression

Given a series of input/output pairs:

$$\langle d_i, c \rangle$$

For each observation text d_i , We represent d_i by a feature vector $X = [x_1, x_2, \dots, x_n]^T$, where x_i can be word count, word frequency etc., and n denotes the vocabulary size.

We then compute an output: a predicted class $\hat{c} \in \{0,1\}$

That is, the binary logistic regression classifier can be regarded as a function f , mapping the features to the class c

$$\hat{c} = f(x_1, x_2, \dots, x_n)$$

Binary Classification in Logistic Regression

Since we introduced the use of **Bag of Words** to represent documents, the **features for logistic regression are the words** themselves. Given that words are associated with different classes, the weight w_i represents the importance of each input feature to the classification decision.

For example, words can be positive, providing evidence that the instance being classified belongs to the positive class, or negative, providing evidence that the instance belongs to the negative class.

Binary Classification in Logistic Regression

For feature x_i , the weight w_i indicates how important x_i is to the classification decision.

- x_i = "review contains 'awesome'": $w_i = +10$
- x_j = "review contains 'abysmal'": $w_j = -10$

We sum up all the weighted features and an additional bias b :

$$z = b + \sum_{i=1}^n w_i x_i$$

The weights can be regarded as a vector $\mathbf{W} = [w_1, w_2, \dots, w_n]^T$. It gives:

$$z = \mathbf{W}^T \mathbf{X} + b$$

If z is high, we say $\hat{c} = 1$; if low, then $\hat{c} = 0$.

Binary Classification in Logistic Regression

We need to formalise "high sum". We desire a principled classifier that provides us with a probability, similar to the approach used by Naïve Bayes. Specifically, we seek a model that can inform us:

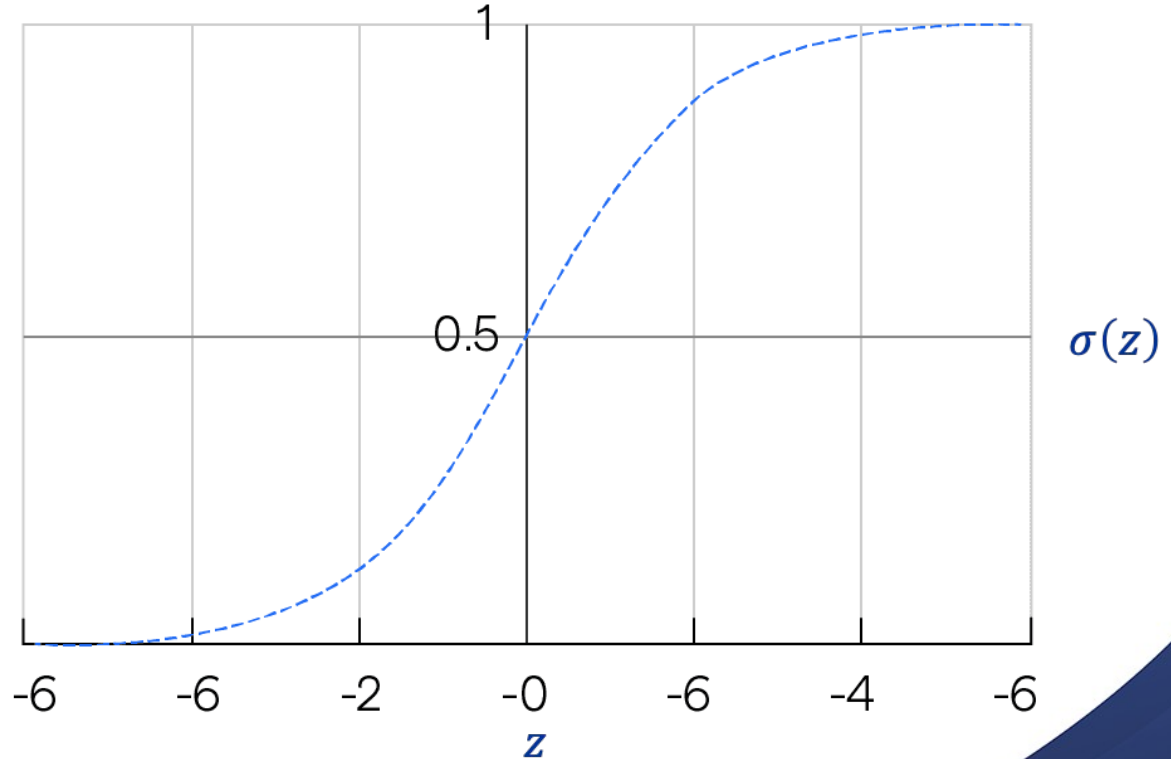
$$\begin{aligned} P(\hat{c} = 1|X, W) \\ P(\hat{c} = 0|X, W) \end{aligned}$$

So far, z is a number, not a probability yet. Thus, we introduce Sigmoid function (denoted by σ) that maps any real-valued number into a value between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Binary Classification in Logistic Regression

Thus, it gives the final prediction:

$$P(y = 1|X, W) = \sigma\left(b + \sum_{i=1}^n w_i x_i\right) = \sigma(W^T X + b)$$

and,

$$\begin{aligned} P(y = 0|X, W) &= 1 - P(\hat{c} = 1|X, W) \\ &= 1 - \sigma(W^T X + b) \\ &= \sigma(-W^T X - b) \end{aligned}$$

NB: $1 - \sigma(x) = \sigma(-x)$

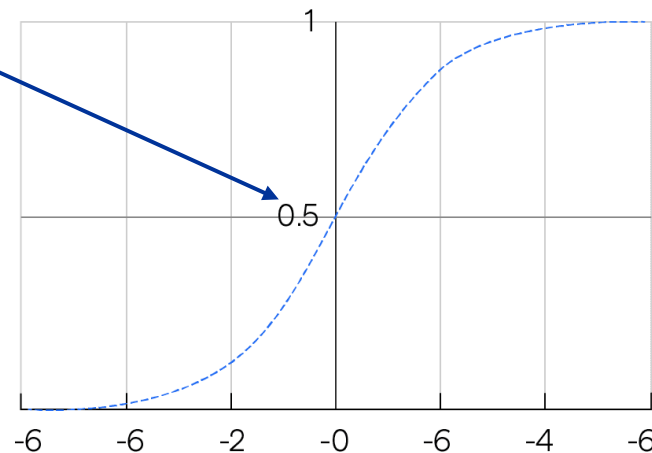
Binary Classification in Logistic Regression

A decision boundary is a hypersurface that separates the feature space into distinct regions, each corresponding to a different class label in a classification problem.

The decision boundary of a sigmoid is 0.5, distinguishing between different class labels.

So,

$$\hat{c} = f(X) = \begin{cases} 1 & \text{if } P(y = 1|X, W) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$



Multinomial Logistic Regression

Because the output of the sigmoid function is a real number, a classifier based on the sigmoid function is necessarily a binary classifier. $y = \sigma(X), X \in \mathbb{R}^n, y \in \mathbb{R}$

A multinomial classifier requires replacing the sigmoid function with the **softmax** function. The latter converts a vector of real numbers into a probability distribution over multiple classes. $Y = \text{softmax}(X), X \in \mathbb{R}^n, Y \in \mathbb{R}^m$

Thus, in the weighting formular, **W** becomes a matrix that records the weight ($w_{i,j}$) of each feature (x_j) for each classification (c_i). **Z, b** becomes a vector as well

$$Z = WX + b$$

Finally:

$$\begin{bmatrix} P(c_1|X) \\ P(c_2|X) \\ \vdots \\ P(c_m|X) \end{bmatrix}_{W,b} = \text{softmax}(WX + b)$$

Logistic Regression Example

Movie Review Example

Suppose we are doing binary sentiment classification on movie review text, and we would like to know whether to assign the sentiment class + or – to a review document doc.

We'll represent each input observation by the 6 manually designed features $x_1, x_2, \dots, x_6$ of the input shown in the following table.

Var	Definition
x_1	count(positive lexicon words $\in$ doc)
x_2	count(negative lexicon words $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	$\ln(\text{word count of doc})$

Movie Review Example

Consider the given document:

It's **hokey**. There are virtually **no** surprises , and the writing is **second-rate**. So why was it so **enjoyable**? For one thing , the cast is **great**. Another **nice** touch is the music . **I** was overcome with the urge to get off the couch and start dancing . It sucked **me** in , and it'll do the same to **you**.

Var	Definition	
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	$\ln(\text{word count of doc})$	$\ln(66)=4.19$

Movie Review Example

We also use human estimated weights: $W = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]^T$ and $b = 0.1$.

Thus,

$$\begin{aligned}P(+|X) &= \sigma(W^T X + b) \\&= \sigma(2.5 \times 3 - 5.0 \times 2 - 1.2 \times 1 + 0.5 \times 3 + 0.2 \times 0 + 0.7 \times 4.19 + 0.1) \\&= \sigma(0.833) \\&= 0.7 \\P(-|X) &= 1 - P(+|X) \\&= 0.3\end{aligned}$$

Because $0.7 > 0.5$, the text should be classified as **positive**.

Supervised Classification

Learning the Weights

In our classifier, we have been using a set of weights $\mathbf{W}$, b as feature coefficients. Up until now, these weights have been estimated manually, which is not sufficiently accurate.

Let us introduce a machine learning approach to determine the optimal weights.

$$P(y = 1 | \mathbf{X}, \mathbf{W}) = \sigma \left(b + \sum_{i=1}^n w_i x_i \right) = \sigma(\mathbf{W}^T \mathbf{X} + b)$$

to learn

Supervised Classification

Supervised classification is a machine learning task where a model is trained on a **labelled dataset** to predict the class labels of new, unseen instances.

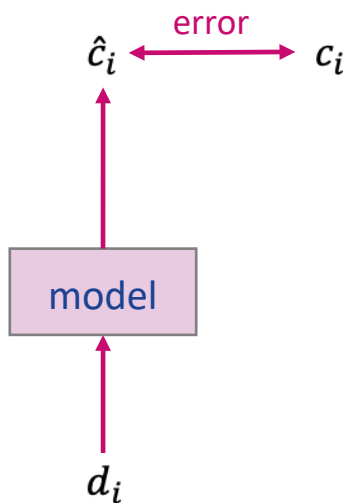
Logistic regression is an instance of supervised classification.

Learning the weights requiring a **labelled corpus**, containing the pairs of text d_i with the correct label c_i .

$$\langle d_i, c_i \rangle \in \text{corpus}$$

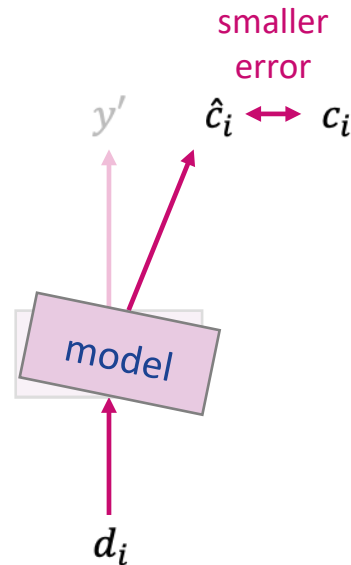
Model Training Process

Given an untrained model, the parameters are initially arbitrary, leading to arbitrary model outputs. We refer to the model's output as the predicted value $\hat{c}_i$ and the desired output as the true value c_i .



The training process aims to **optimize** the model's parameters based on the error, making the predicted value closer to the true value.

This process is repeated multiple times until the error no longer decreases.



Model Training

There are two questions for the training process:

- How to measure the error between the predicted value and the true value (use the [loss function](#)).
- How to [optimise](#) the parameters according to the loss?

Cross-entropy (recall)

Cross-entropy is a measure used in information theory and machine learning to **quantify the difference between two probability distributions**.

It calculates the average number of bits needed to identify an event from a set of possibilities, comparing the predicted distribution to the actual distribution.

In simpler terms, it tells us how well a model's predictions match the true outcomes, with lower cross-entropy indicating better predictions (Note the negative sign in the definition of cross-entropy).

$$\text{CrossEntropy}(Q, P) = - \sum_{v \in \text{vocab}} Q(v) \cdot \log_2 P(v) = - \frac{1}{N} \sum_{w \in \text{corpus}} \log_2 P(w)$$

We are going to use Cross-entropy to measure the error.

Cross-entropy for Binary Classification

Cross-entropy is used as the loss function for classifiers because it measures the difference between the true label distribution and the predicted label distribution.

It effectively penalises predictions that are confident but incorrect, encouraging the model to output probabilities that are closer to the actual labels.

This property makes it particularly suitable for classification tasks, ensuring that the model improves its accuracy and certainty in its predictions.

Cross-entropy for Binary Classification

The binary classifier predicts the probabilities of two classes:

$$P(c_1|X) = \hat{y} \quad \text{and} \quad P(c_2|X) = 1 - \hat{y}$$

where $\hat{y} = \sigma(-W^T X - b)$.

Let the ground true label for X is y e.g. $y = 0$ if c_1 , $y = 1$ if c_2 . y and $\hat{y}$ can be regarded as two probabilities, so that we can plug y and $\hat{y}$ into cross-entropy. By considering both it gives the measure of the error from $\hat{y}$ to y .

$$\begin{aligned} \mathcal{L}(\hat{y}, y) &= \text{CrossEntropy}(\hat{y}, y) = - \sum_{c_i} P(c_i) \log_2 P(\hat{c}_i) \\ &= -P(c_1) \log_2 P(\hat{c}_1) - P(c_2) \log_2 P(\hat{c}_2) \\ &= -y \log_2 \hat{y} - (1 - y) \log_2 (1 - \hat{y}) \end{aligned}$$

Cross-entropy for Multinomial Classification

The multi-class classifier predicts the probabilities of multi classes:

$$\begin{bmatrix} P(c_1|X) \\ P(c_2|X) \\ \vdots \\ P(c_m|X) \end{bmatrix}_{W,b} = \text{softmax}(WX + b)$$

Although Multinomial Classification predicts multiple classes, the correct answer **c must be a single class**. Thus, when Cross-entropy measure each class

$$\begin{aligned} \mathcal{L}(\hat{y}, y) &= \text{CrossEntropy}(\hat{y}, y) = - \sum_{c_i} P(c_i) \log_2 P(\hat{c}_i) \\ &= -P(c_1) \log_2 P(\hat{c}_1) - P(c_2) \log_2 P(\hat{c}_2) - \dots - P(c_m) \log_2 P(\hat{c}_m) \end{aligned}$$

Among red probabilities, only one is 1, while the others are all 0.

Cross-entropy for Multinomial Classification

Therefore, the cross-entropy for classification (both binary and multinomial) can be simplified, by only considering the predict probability $\hat{c}_c = c$ alone with the ground true label c .

$$\begin{aligned}\mathcal{L}(\hat{y}, y) &= \text{CrossEntropy}(\hat{y}, y) = - \sum_{c_i} P(c_i) \log_2 P(\hat{c}_i) \\ &= -\cancel{P(c_1)}^0 \log_2 P(\hat{c}_1) - \dots - \cancel{P(c_c)}^1 \log_2 P(\hat{c}_c) - \dots - \cancel{P(c_m)}^0 \log_2 P(\hat{c}_m) \\ &= -\log_2 P(\hat{c}_c)\end{aligned}$$

Gradient Descent

Our goal with gradient descent is to **find the optimal weights: minimise the loss** function we've defined for the model. Here, we use θ to denote all the learnable parameters in the classifier (i.e. W and b in our example).

So the goal is to find the set of weights which minimises the loss function, averaged over all examples:

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\hat{y}, y) = \mathcal{L}(f_{\theta}(X), y)$$

The formular means we want a $\theta = \hat{\theta}$, which lets the value of $\mathcal{L}(\hat{y}, y)$ as minimum as possible.

Gradient Descent

We aim to iteratively adjust θ to approach the $\hat{\theta}$, rather than directly obtaining the $\hat{\theta}$ through solving.

According to the loss, we can calculate **how to adjust the weights** (i.e. θ) to let the predicted value $\hat{y}$ approach to y , though the gradient (i.e. the differentials).

The **derivative of the loss $\mathcal{L}$ with respect to the weights θ** expresses how the value of the loss changes as the weights change. In other words, it indicates how to adjust the weights to reduce the loss. The chain rule gives:

$$\nabla_{w,b} = \frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \sigma} \cdot \frac{\partial \sigma}{\partial \theta}}_{\text{chain rule}} = \dots = \begin{bmatrix} (\hat{y} - y)x_1 \\ (\hat{y} - y)x_2 \\ \hat{y} - y \end{bmatrix}$$

Diagram illustrating the chain rule for the gradient of the loss $\mathcal{L}$ with respect to the weights w_1 , w_2 , and b . The gradient vector is shown as a column vector with three components: $(\hat{y} - y)x_1$, $(\hat{y} - y)x_2$, and $\hat{y} - y$. Arrows point from the partial derivatives $\frac{\partial \mathcal{L}}{\partial w_1}$, $\frac{\partial \mathcal{L}}{\partial w_2}$, and $\frac{\partial \mathcal{L}}{\partial w_b}$ to their respective components in the vector.

This process depends on the loss function and model formular

Gradient Descent

NB: When calculating the gradient, we have already computed the loss, which means that each parameter in the formula has a specific value. Consequently, the gradient can always be computed as a **concrete numerical value** rather than remaining in the form of an abstract formula.

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \frac{\partial \mathcal{L}}{\partial b} \end{bmatrix} = \begin{bmatrix} (\hat{y} - y)x_1 \\ (\hat{y} - y)x_2 \\ \hat{y} - y \end{bmatrix} = \begin{bmatrix} \text{num}_1 \\ \text{num}_2 \\ \text{num}_3 \end{bmatrix}$$

Parameter Optimisation

Finally, once we have calculated the gradient, we use it to **update the model parameters**.

To reduce the loss, we should adjust the model parameters in the direction opposite to that indicated by the gradient. That is:

$$\theta \leftarrow \theta - \eta \nabla_{w,b}$$

For example:

$$w_1 \leftarrow w_1 - \eta \frac{\partial \mathcal{L}}{\partial w_1}$$

In the equations, η is the learning rate, which controls the size of the steps taken during gradient descent to update the model parameters, balancing between convergence speed and stability (usually $0.0001 < \eta < 0.1$).

Summary of the Model Training Process

The model training process can be roughly described as follows.

1. **Initialize Parameters:** Randomly set initial values for model parameters.
2. **Forward Propagation:** Compute predictions using the current parameters.
3. **Compute Loss:** Calculate the **loss** function to measure the difference between predictions and true values.
4. **Backward Propagation:** Calculate the **gradient** of the loss with respect to each parameter.
5. **Update Parameters:** Adjust parameters in the direction opposite to the gradient, scaled by the learning rate.
6. **Repeat:** Iterate steps 3-6 until convergence criteria are met.

Conclusion

Logistic Regression: A statistical method for binary classification, modelling the probability of an outcome based on input features.

Discriminative Classifier: Focuses on estimating the decision boundary between classes rather than modelling individual distributions.

Sigmoid Function: Converts weighted sums of features into probabilities between 0 and 1, with a decision boundary at 0.5.

Model Training: Utilises supervised learning to optimise weights using cross-entropy as the loss function, improved via gradient descent.

Applications: Widely used in NLP tasks like sentiment analysis and classification problems, serving as a foundation for more complex models.